Pizza Sales Analysis

Practice Questions & Solutions

Q1. Retrieve the total number of orders placed.

Solution:

```
SELECT
    COUNT(*) AS total_orders
FROM orders;
```

Q2. Calculate the total revenue generated from pizza sales.

Solution:

```
SELECT
    SUM(od.quantity * p.price) AS total_revenue
FROM order_details od JOIN pizzas p ON od.pizza_id = p.pizza_id;
```

Q3. Identify the highest-priced pizza.

Solution:

```
SELECT
    pt.name,
    p.pizza_id,
    pt.category,
    pt.ingredients,
    p.price
FROM pizza_types pt JOIN pizzas p ON pt.pizza_type_id =
p.pizza_type_id
ORDER BY p.price DESC
LIMIT 1;
```

Q4. Identify the most common pizza size ordered.

Solution:

```
SELECT
    p.size,
    COUNT(o.order_id) AS total_orders
FROM order_details od JOIN pizzas p ON od.pizza_id = p.pizza_id
```

```
                        JOIN orders o ON od.order_id = o.order_id
GROUP BY 1
ORDER BY 2 DESC
LIMIT 1;
```

Q5. List the top 5 most ordered pizza types along with their quantities.

Solution:

```
SELECT
    pt.name,
    pt.category,
    pt.ingredients,
    SUM(od.quantity) AS total_quantity
FROM order_details od JOIN pizzas p ON od.pizza_id = p.pizza_id
                      JOIN orders o ON od.order_id = o.order_id
                      JOIN pizza_types pt ON pt.pizza_type_id =
p.pizza_type_id
GROUP BY 1,2,3
ORDER BY 4 DESC
LIMIT 5;
```

Q6. Join the necessary tables to find the total quantity of each pizza category ordered.

Solution:

```
SELECT
    pt.category,
    SUM(od.quantity) AS total_quantity
FROM order_details od JOIN pizzas p ON od.pizza_id = p.pizza_id
                      JOIN orders o ON od.order_id = o.order_id
                      JOIN pizza_types pt ON pt.pizza_type_id =
p.pizza_type_id
GROUP BY 1
ORDER BY 2 DESC;
```

Q7. Determine the distribution of orders by hour of the day.

Solution:

```
SELECT
    DATE_PART('HOUR',TIME) AS hours,
    COUNT(order_id) AS total_orders
FROM orders
GROUP BY 1
ORDER BY 2 DESC;
```

Q8. Join relevant tables to find the category-wise distribution of pizzas.

Solution:

```
SELECT
    category,
    COUNT(name) AS number_pizza
FROM pizza_types
GROUP BY 1
ORDER BY 2 DESC;
```

Q9. Group the orders by date and calculate the average number of pizzas ordered per day.

Solution:

```
WITH daily_avg AS
(
SELECT
    o.date,
    SUM(od.quantity) AS total_quantity
FROM orders o JOIN order_details od ON od.order_id = o.order_id
GROUP BY 1
ORDER BY 1
)
SELECT
    FLOOR(AVG(total_quantity)) AS average_order
FROM daily_avg;
```

Q10. Determine the top 3 most ordered pizza types based on revenue.

Solution:

```
SELECT
    pt.name,
    SUM(od.quantity * p.price) AS total_revenue
FROM order_details od JOIN pizzas p ON od.pizza_id = p.pizza_id
                      JOIN orders o ON od.order_id = o.order_id
                      JOIN pizza_types pt ON pt.pizza_type_id =
p.pizza_type_id
GROUP BY 1
ORDER BY 2 DESC
LIMIT 3;
```

Q11. Calculate the percentage contribution of each pizza type to total revenue.

Solution:

```
WITH cte AS
(
SELECT
    pt.name,
    od.quantity * p.price AS revenue
FROM order_details od JOIN pizzas p ON od.pizza_id = p.pizza_id
                      JOIN orders o ON od.order_id = o.order_id
                      JOIN pizza_types pt ON pt.pizza_type_id =
p.pizza_type_id
)
SELECT
    name,
    ROUND(((SUM(revenue) * 100.0) / (SELECT SUM(revenue) FROM
CTE)),2) AS "% Contribution"
FROM cte
GROUP BY 1;
```

Q12. Analyze the cumulative revenue generated over time.

Solution:

```
WITH cumulative_revenue AS
(
SELECT
    o.date,
    SUM(od.quantity * p.price) AS total_revenue
FROM order_details od JOIN pizzas p ON od.pizza_id = p.pizza_id
                      JOIN orders o ON od.order_id = o.order_id
                      JOIN pizza_types pt ON pt.pizza_type_id =
p.pizza_type_id
GROUP BY 1
ORDER BY 1
)
SELECT
    *,
    SUM(total_revenue) OVER (ORDER BY date ASC) AS running_total
FROM cumulative_revenue;
```

Q13. Determine the top 3 most ordered pizza types based on revenue for each pizza category.

Solution:

```
WITH top3 AS
(
SELECT
    pt.category,
    pt.name,
    SUM(od.quantity * p.price) AS total_revenue
FROM order_details od JOIN pizzas p ON od.pizza_id = p.pizza_id
                      JOIN orders o ON od.order_id = o.order_id
                      JOIN pizza_types pt ON pt.pizza_type_id =
p.pizza_type_id
GROUP BY 1,2
ORDER BY 1 ASC, 3 DESC
),
ranks AS
(
```

```
SELECT
    *,
    DENSE_RANK() OVER (PARTITION BY category ORDER BY total_revenue
DESC) AS rno
FROM top3
)
SELECT
    *
FROM ranks
WHERE rno <= 3;
```